The API

Trunk section: §4 The running parts.

Every route of the new desk in one place, by domain, and what it replaces. The domain leaves describe each route's behaviour in their contracts sections.

Built on: the three legacy APIs: screeners (each tool's routes, plus T1's Manage tools), qp (53 routes and a WebSocket), journal (29 routes, of which 14 had no caller).

---

1. Conventions

Rule	Meaning
one origin	every route is behind the front door: `/api/…` on desk-web, `/quant/api/…` on quant
login	every route needs the session cookie, except `/login`, `/logout` and `/health` (which answers only up or down)
same origin	a request that changes anything must carry the desk's own `Origin`; the cookie is `Secure`, `HttpOnly`, `SameSite=Lax`
JSON	requests and answers; exports also as CSV or Parquet where stated
errors	`4xx` with `{ error, reasons[] }` for a refused request (every reason listed); `5xx` only for a fault, logged with its stack
as of	read routes over history take `day` or `at`; nothing returns data from after it
no secrets	no answer ever contains a key or a secret
every route has a caller	a page, a job or a test; a route nothing calls is not built (legacy: 14 of the journal's 29)

---

2. desk-web routes

Platform (PLATFORM.md)

Method	Route	Does
GET, POST	`/login`	the login page; sign in
POST	`/logout`	sign out (with “everywhere”)
GET	`/health`	up or down (no login)
GET	`/api/health`	every health signal, with fixes
POST	`/api/health/run`	run every check now
GET	`/api/jobs`	every job: schedule, last and next runs
PATCH	`/api/jobs/:job`	schedule, enabled (checked)
POST	`/api/jobs/:job/run`	run now
GET	`/api/jobs/:job/runs`	run history
GET, PATCH	`/api/settings`	settings (checked, versioned)
POST	`/api/settings/keys/:name/test`	test a key (the value is never returned)
GET	`/api/backups`	nightly backups and their verification

Sources (SOURCES.md)

Method	Route	Does
GET	`/api/sources`	adapters: calls, failures, last errors
GET	`/api/cache`	the bar cache: size, free disk
POST	`/api/cache/sweep`, `/api/cache/clear`	trim, clear

Tools and screeners (TOOLS-AND-SCREENERS.md)

Method	Route	Does
GET, POST	`/api/tools`	list; add (checked)
PATCH	`/api/tools/:tool`	edit (checked: capture vs windows)
POST	`/api/tools/:tool/{pause,resume,archive,start,delete,restore}`	lifecycle
GET, POST	`/api/tools/:tool/screeners`	list; add
PATCH	`/api/tools/:tool/screeners/:key`	edit (a new version)
GET	`/api/tools/:tool/screeners/:key/versions`	version history
POST	`/api/screeners/test`	run a definition now, unsaved (`count` , `totalCount` , sample)
POST	`/api/screeners/preview-mirror`	the mirror, unsaved
GET	`/api/screeners/meta`	columns (from the adapter), operators, post-scan tests
GET	`/api/tools/:tool/scans?day=`	scan reports

Card (CARD.md)

Method	Route	Does
GET	`/api/tools/:tool/cards?day=&filters`	the tool’s cards (search, screener, news, catalyst, aligned, sort)
GET	`/api/cards/:ticker?day=`	one card: every family with status and as-of
GET	`/api/cards/:ticker/canslim?day=`	the CANSLIM panel
POST	`/api/tools/:tool/cards/:ticker/bias`	manual bias (re-scores)

Market (MARKET.md)

Method	Route	Does
GET	`/api/market/state?at=`	the market state
GET	`/api/market/history?day=`	every state of a day
GET	`/api/market/groups?day=`	the group ranking
POST	`/api/market/hot/reset`	reset hot state (logged)

Registers (REGISTERS.md)

Method	Route	Does
GET	`/api/tools/:tool/registers/:reg/dates`	the days a register has (R1, R2, R3, R4, shortlist)
GET	`/api/tools/:tool/registers/:reg/:day?format=csv|json|legacy`	the register
POST, DELETE	`/api/tools/:tool/shortlist/:ticker`	star, unstar (the card's ☆/★ reads the shortlist)
GET	`/api/shortlist?day=`	every tool's shortlist
POST	`/api/tools/:tool/registers/R4/upload`	upload external rows (kept apart)

Learning (LEARNING.md)

Method	Route	Does
GET	`/api/tools/:tool/models`	model versions, metrics, publish decisions
GET	`/api/tools/:tool/scorecard?entry=A|B`	every screener's numbers and verdict
GET	`/api/compare?entry=A|B`	against the benchmark and the mirrors

Journal (JOURNAL.md)

Method	Route	Does
GET	`/api/journal/trades?filters`	trades
GET	`/api/journal/trades/:id`	a trade with fills and snapshot
POST	`/api/journal/trades`	hand entry
PATCH	`/api/journal/trades/:id`	stop, target, fees, setup, notes, tags
DELETE	`/api/journal/trades/:id`	delete
POST	`/api/journal/import`	a broker file → the import report
GET	`/api/journal/stats?filters`	every number of the page, server-computed
GET, POST	`/api/journal/accounts`	accounts (no secrets)
DELETE	`/api/journal/accounts/:id`	remove (trades stay)
POST	`/api/journal/accounts/:id/{test,fetch}`	test the key; fetch now
GET, POST, PATCH	`/api/journal/setups`	setups
GET, PUT	`/api/journal/notes/:day`	day notes

---

3. quant routes (`/quant/api/…`)

The legacy qp routes carry over (legacy/qp/API.md): chart data and the live WebSocket (`/ws/live`), the library and approvals, strategies (save, load, evaluate, test a rule, explain, explain a scan), backtests (run, status, stop, delete, report, stats, CSV, journal CSV), the register browser, alerts (start, stop, status, recent), print sheets (R1 day and swing sheets, their CSVs, the cards CSV, pairs print and CSV), the feed check and cache routes. Changes:

Change	Detail
registers and facts	read from the database as of a day, not over HTTP from tool ports
O'Neil routes (`/api/oneil/*`)	removed: the facts come from the research worker, read through desk-web's card and market routes
alerts	history stored; `GET /quant/api/alerts/history` added
data check	moved to desk-web's health
feed defaults	no route defaults to a feed of its own

4. What is not carried over

Legacy route(s)	Why
`/api/market/oneil/*` on every tool (asking qp per card)	facts are read from the fact store
`/api/warehouse/*` served by the archive on stopped tools' ports	registers are one set of tables
the journal's templates, context, compute, compute-all, backfill-to-grading, its own metrics and calendar	no screen used them; replaced by the journal's stats and snapshots
scorer processes' routes, including the dead per-setup grading routes	models are applied in-process
`/api/tools/manage` on T1 only	`/api/tools` on desk-web
per-tool `/api/monitor` , `/api/datacheck`	`/api/jobs` , `/api/health`